How to Answer Secure and Private SQL Queries?

Qiyao Luo*
OceanBase, Ant Group
luoqiyao.lqy@antgroup.com

Quanqing Xu*
OceanBase, Ant Group
xuquanqing.xqq@oceanbase.com

Chuanhui Yang†
OceanBase, Ant Group
rizhao.ych@oceanbase.com

***Abstract*—In the era of big data, the ability to process and analyze large volumes of data is critical for decision-making, marketing and sales, healthcare and scientific research, etc. However, this capability also brings significant challenges related to data security and privacy. Secure and private query processing is essential to address these challenges. It ensures the protection of private and confidential data, compliance with stringent regulatory requirements, and the maintenance of user trust. Additionally, it enables safe data sharing and collaborative analysis while preserving the utility of the data. By incorporating advanced cryptographic and privacy techniques, secure and private query processing can defend against sophisticated cyber threats. This tutorial highlights the importance of integrating robust security and privacy measures into query processing to build trustworthy database systems. It reviews current systems and protocols that achieve these goals and discusses future directions for easy-to-use query processing under secure and private protection.**

***Index Terms*—security, privacy, query processing, secure computation, differential privacy**

I. Introduction

Database management systems (DBMSs) are the backbone of numerous applications ranging from financial transactions and healthcare management to social media and e-commerce. SQL queries are the primary mechanism for interacting with and managing data within a DBMS. In the era of big data, the growing volume of data has driven the need for *outsourced computation*, where data is processed on public cloud services. Furthermore, the distributed nature of data across multiple parties underscores the importance of *collaborative analysis*. However, both these settings are fraught with risks of data breaches and privacy violations, which can have severe consequences. Additionally, beyond the computation stage, releasing query results also poses significant risks, particularly from differential attacks, reconstruction attacks and the exploitation of auxiliary information. Therefore, there is a growing need to integrate robust security and privacy features into answering SQL queries.

Traditional protection methods involve encrypting sensitive data on disk, restricting unauthorized access, and masking data before sharing. However, these measures are increasingly insufficient in the face of advanced hacking techniques. Fortunately, the development of recent privacy-preserving technologies makes it possible to process data in an "available but invisible" manner. That is, users are analyzing the data using SQL queries as usual, while the executions avoid exposing or revealing the actual contents. These technologies span various domains, including cryptography (e.g., *secure multi-party computation*, MPC), hardware (e.g., *trusted execution environments*, TEE), and statistics (e.g., *differential privacy*, DP). Although initially introduced as theoretical concepts in the 1970s, it is only in the last decade that these techniques have been evolved into practical solutions, driven by advancements in computing infrastructure and protocol design. Many companies are actively researching and implementing these advanced protections. For example, Azure offers "Azure Confidential Databricks", Alibaba Cloud provides "Always-Confidential Database", and Ant Group has developed "Secure Collaborative Query Language" (SCQL).

In this tutorial, we first review the current state-of-the-art privacy-preserving techniques, including secure computation and differential privacy, to protect data privacy in intermediate computations and result release respectively. Next, we go through the protection under the context of query processing. We start by introducing how to design a secure relational operator, and then compose them to build a secure query processor. Additionally, we consider incorporating differential privacy to protect the results and accelerate the process. We review the current systems and algorithms, finding their protection capabilities and trade-offs. Finally we will outline the future directions of secure and private query processing.

Length: The tutorial is designed to last 1.5 hours and is structured into four parts:

1) **Introduction and Preliminaries (20 mins)**: This part provides a comprehensive introduction to the various security and privacy techniques used in query processing;
2) **Secure Query Processing (30 mins)**: This part delves into secure query processing in details, starting with basic functionalities, then progressing to relational operators and complex SQL query processing procedures;
3) **Differential Privacy (30 mins)**: This part focuses on differential privacy from three perspectives: protecting query results, accelerating query processing, and generating DP noise under MPC;
4) **Practical Directions (10 mins)**: The final part presents three directions for implementing secure and private query processing in practical scenarios.

Target Audience: The target audience for this tutorial includes 1) database researchers who are interested in understanding and advancing the state of secure and private

*These authors contributed equally to this work. †Chuanhui Yang is the corresponding author.

query processing; 2) privacy experts who want to explore the intersection of cryptography and database query processing; and 3) industrial engineers who are looking to discuss the challenges and solutions related to deploying security and privacy features in DBMSs.

This tutorial does not require prior knowledge of cryptography as relevant cryptographic primitives will be introduced as black boxes. We will compare secure and private query processing with traditional plaintext database systems to provide a clear understanding of the challenges.

II. Tutorial Outline

A. Introduction and Preliminaries

We begin by motivating the critical need for security and privacy in the current era of big data. We distinguish between security which focuses on protecting data during computation; and privacy which ensures that individual data remains confidential when disclosed. We also discuss the current legal and commercial requirements that drive the need for security and privacy measures.

Next we introduce secure computation and differential privacy respectively:

- **Secure Computation**:
  - We primarily focus on three popular techniques: MPC, TEE, and fully homomorphic encryption (FHE). We will discuss their benefits, weaknesses, and typical use cases.
  - Additionally, we briefly cover other relevant techniques such as zero-knowledge proof (ZKP), private information retrieval (PIR), symmetric searchable encryption (SSE) and order-preserving encryption (OPE).
  - A central concept in secure computation is the oblivious algorithms, which ensure that the computations are performed in a way that does not reveal any information. We will explore the implementation of these algorithms, including the use of circuits as a generic solution and specific protocols designed for particular settings.
- **Differential Privacy**:
  - We introduce different models of DP: central model (CDP), local model (LDP) and shuffle model (SDP), and their associated trust assumptions, along with the contexts in which they are most applicable.
  - We then explore popular mechanisms such as the Laplace and Gaussian mechanisms for CDP, randomized response for LDP, the shuffle amplification for SDP, and the post-processing and composition theorems.

Finally, we will define the protection in the context of query processing, where queries can vary from simple statistical aggregations to complex SQL queries. This will provide a comprehensive understanding of how security and privacy techniques can be integrated into various types of data queries.

Overview. Typically, the query processing in a data lifecycle can be divided into three stages: data collection, data processing, and data release. Each stage involves the application of different techniques to ensure varying levels of protection. An overview of this process is provided in Figure 1.

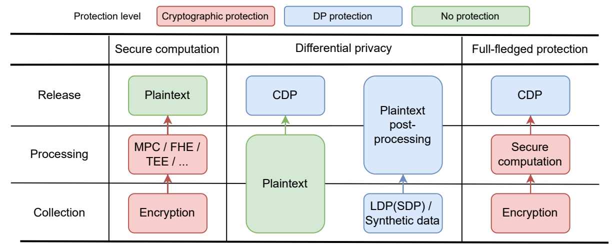

Fig. 1: The three stages of query processing

B. Secure Query Processing

In this part, we explore secure query processing through the modular approach, as illustrated in Figure 2. It involves breaking down complex problems into simpler subproblems which makes secure protocol designing efficient.

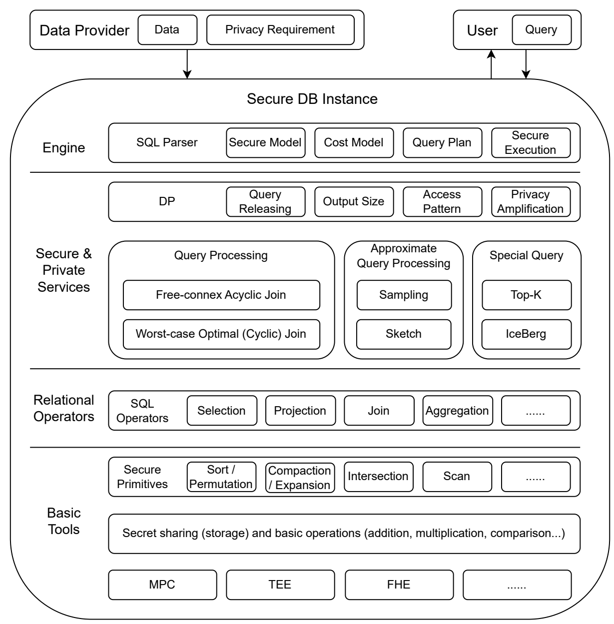

Fig. 2: An overview of secure query processing

We first go through the structure in a bottom-up manner to get an overview of secure query processing and introduce the benefits of modular approach.

Then we go deep to core components: We review the relational operators [1], a class of operations that can be directly expressed as SQL expressions. When executed within a plaintext database, these operators come with running time guarantees. It is also meaningful in secure computation. We summarize the implementation details of each operator (how these operators are composed by secure primitives), including its circuit complexity [2] and efficient solutions in specific secure models [3]–[5] (see Table I).

TABLE I: Summary of relation operators, where n denotes the total input size and m denotes the output size

Operator	SQL Query	Plaintext	Circuit	Efficient Secure Solution
Selection($\sigma_f(R)$)	`SELECT * FROM R WHERE f;`	$O(n)$	$O(n)$	$O(n)$
Projection($\pi_{\boldsymbol{E}}(R)$)	SELECT $\boldsymbol{E}$ FROM R;	$O(n)$	$O(n)$	$O(n)$
Order-By($\tau_{\boldsymbol{A}}(R)$)	SELECT * FROM R ORDER BY $\boldsymbol{A}$ DESC;	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log n)$
Union($R_1 \cup R_2$)	SELECT * FROM R_1 UNION R_2	$O(n)$	$O(n)$	$O(n)$
Join($R_1 \bowtie R_2$)	SELECT * FROM R_1 NATURAL JOIN R_2;	$O(n+m)$	$O(n \log^2 n + m \log^2 m)$	$O(n \log n + m)$
Semi-Join($R_1 \ltimes R_2$)	SELECT * FROM R_1 WHERE EXISTS (SELECT 1 FROM R_2 WHERE $R_2.\boldsymbol{K} = R_1.\boldsymbol{K}$);	$O(n)$	$O(n \log^2 n)$	$O(n \log n)$
Group-By Aggregation($\pi_{\boldsymbol{E}}^{\oplus(\boldsymbol{A})}(R)$)	SELECT $\boldsymbol{E}$, $\oplus(\boldsymbol{A})$ FROM R GROUP BY $\boldsymbol{E}$;	$O(n)$	$O(n \log^2 n)$	$O(n \log n)$

The join operator is particularly challenging. We explore various join types, including PK-PK join (a variant of private set intersection with payload) [6]–[8], PK-FK join [9], [10], natural join [11]–[13], outer join [9], and semi-join [5], [14], introducing standard and efficient oblivious algorithms for these operations. Specifically, we show the improvement line of natural join operator from nested-loop-join [15], [16] to sort-merge-join [11]–[13], [17] with the development of secure primitives (e.g., MPC sorting [18]–[20], TEE sorting [21], [22]); and to the index nested-loop join by using oblivious index [23], [24] under TEE.

Next we consider answer complex SQL queries using secure relational operators. Many research stops at the operator layer. We want to point out that the straightforward composition is not secure - it will reveal more information than we hope. Only a few works study this. [5], [14], [23] solve the free-connex join-aggregate queries under 2PC, 3PC and TEE model respectively, which is the largest class of queries that can be solvable in linear time (i.e., $O(n+m)$ where n, m are the input and output size respectively) in plaintext. Free-connex join-aggregate queries are the most practical type of queries that one can hope for in secure computation. [2], [25] study the worst-case optimal join problems (e.g., the cyclic queries) theoretically and presents a circuit-based and ORAM solution respectively.

To mitigate the computational expense of cryptographic operations (which is always more than 1000x larger than that of plaintext operations), approximate query processing is valuable. It includes sampling [4], [26], [27] and sketching methods [28]–[30].

There are also other secure protocols which focus on special query types, such as top-k query (with an "order-by limit k" clause) [1] and ice-berg query (with a "group-by having" clause) [31].

Finally, we analyze existing secure systems using different secure computation techniques.

- Encryption-based: CryptDB [32], HE^{3}DB [33], JXL [34], [35], etc.;
- TEE-based: TrustedDB [36], EnclaveDB [37], Opaque [38], ObliDB [16], etc.;
- MPC-based: SMCQL [15], SECYAN [14], Secrecy [3], Senate [8], Conclave [39], etc.

C. Differential Privacy

1) Answering Queries under DP: We begin by examining the simple counting problem, also known as select-aggregation (SA) queries, under the tuple-level differential privacy (tuple-DP). In this context, neighboring datasets differ by at most one tuple. They are straightforward in central-DP [40], but become significantly more interesting and challenging in distributed settings. These problems are known as frequency estimation and its extensions: range counting and heavy hitter detection. We review the protocols for local-DP [41]–[43] and shuffle-DP [44]–[47].

Next, we discuss queries involving join operators. These queries are significantly more challenging because a single tuple can now affect multiple join results. This category includes join-aggregation (JA), select-join-aggregation (SJA), and select-project-join-aggregation (SPJA) queries [48]–[51].

We then introduce queries under user-level differential privacy (user-DP), where neighboring datasets can differ by several tuples related to one single user [52]–[54]. Additionally, we address queries under continual observation, which involves periodically answering queries on a dynamic dataset [55]–[57].

We also talk about some hot topics that are relevant to practical scenarios: personalized differential privacy (PDP) and answering multiple queries. PDP [58]–[60] provides customized privacy budget for each tuple. Answering multiple queries [61] allows for the disclosure of numerous queries instead of one under an overall privacy budget.

Finally, we compare some DP systems that can automatically answer SQL queries with DP protection, PINQ [62], FLEX [48], PrivateSQL [52], DOP-SQL [63].

2) Accelerating Query Processing with DP: Despite its primary ability in protecting the query results, DP is also used to accelerate the query processing procedures, albeit with a relaxed level of protection.

a) Output Size: One acceleration is the output size. Instead of padding the output result to the worst-case size for all cases, revealing the real size with an addition of DP noise can be helpful [64].

b) Access Pattern: Another idea is (ε, δ)-differential obliviousness (DO) [65]. Unlike full obliviousness, which requires that access patterns are indistinguishable among all datasets, DO relaxes this requirement by allowing access patterns to satisfy (ε, δ)-DP for neighboring datasets. This relaxation makes oblivious algorithms much more efficient. Relational operators [66]–[68] also benefit from this approach.

3) Secure Noise Generation: We introduce how to generate differentially private noises under secure computation, which is also known as secure sampling for DP. The study [69] focuses on the multi-party setting and introduces different methods: distributed noise generation, uniform transformation and bitwise sampling. Although the first one is much efficient in the three approaches, the latter two can defend against malicious attackers.

D. Future Directions

In the last part, we discuss the open problems in three directions.

1) **Efficiency**: Efficiency is paramount. The adage "time is money" underscores the importance of minimizing computational overhead. A significant barrier to the adoption of security and privacy services is the huge time gap compared to plaintext operations. There are two potential methods for improvement: improving the efficiency of current technologies at both software and hardware levels, and adjusting the protection level to provide a balance that meets users' requirements.
2) **Completeness**: Completeness involves ensuring comprehensive coverage of all functionalities available in plaintext databases. Several areas remain open and challenging, such as transaction management, handling dynamic datasets, query parsing and optimization, and implementing more complex operators with intricate structures. Achieving this will facilitate a seamless transition from plaintext DB systems to secure and private DB systems.
3) **Usability**: Usability, or ease of use, demands that secure applications impose no additional requirements on users. Currently, operating a secure system requires a cryptography expert who understands the security definitions and potential leakages. The challenge is to make these systems accessible to everyone.

III. Presenters

Qiyao Luo is a researcher of OceanBase Lab, Ant Group. His research interests include secure, private, and efficient database algorithms, with a particular focus on secure multi-party computation and differential privacy. He obtained his PhD at the Hong Kong University of Science and Technology.

Quanqing Xu is the technical director of OceanBase Lab, Ant Group. His research interests [70]–[75] primarily include distributed database systems, storage systems and privacy-preserving database systems. He is a Fellow of the IET, a distinguished member of CCF, a senior member of ACM and IEEE.

Chuanhui Yang is the CTO of OceanBase, Ant Group. His research [70]–[75] focuses on database and distributed systems. As one of the founding members, he led the previous architecture design, and technology research and development of OceanBase, realizing the full implementation of OceanBase in Ant Group from scratch.

References

[1] Q. Wang, Q. Luo, and Y. Wang, "Relational algorithms for top-k query evaluation," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, 2024.

[2] Y. Wang and K. Yi, "Query evaluation by circuits," in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2022, p. 67–78.

[3] J. Liagouris, V. Kalavri, M. Faisal, and M. Varia, "SECRECY: Secure collaborative analytics in untrusted clouds," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1031–1056.

[4] Q. Luo, Y. Wang, K. Yi, S. Wang, and F. Li, "Secure sampling for approximate multi-party query processing," *Proceedings of the ACM on Management of Data*, vol. 1, no. 3, 2023.

[5] Q. Luo, Y. Wang, W. Dong, and K. Yi, "Secure query processing with linear complexity," 2024.

[6] P. Mohassel, P. Rindal, and M. Rosulek, "Fast database joins and psi for secret shared data," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, p. 1271–1287.

[7] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, "Efficient circuit-based psi with linear communication," in *Advances in Cryptology – EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2019, p. 122–153.

[8] R. Poddar, S. Kalra, A. Yanai, R. Deng, R. A. Popa, and J. M. Hellerstein, "Senate: A maliciously-secure MPC platform for collaborative analytics," in *Proceedings of the 30th Conference on USENIX Security Symposium*, 2021.

[9] G. Asharov, K. Hamada, R. Kikuchi, A. Nof, B. Pinkas, and J. Tomida, "Secure statistical analysis on multiple datasets: Join and group-by," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, p. 3298–3312.

[10] Q. Luo, Y. Wang, Z. Ren, K. Yi, K. Chen, and X. Wang, "Secure machine learning over relational data," 2021.

[11] S. Krastnikov, F. Kerschbaum, and D. Stebila, "Efficient oblivious database joins," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, p. 2132–2145, 2020.

[12] F. Han, L. Zhang, H. Feng, W. Liu, and X. Li, "Scape: Scalable collaborative analytics system on private database with malicious security," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 1740–1753.

[13] S. Badrinarayanan, S. Das, G. Garimella, S. Raghuraman, and P. Rindal, "Secret-shared joins with multiplicity from aggregation trees," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2022, p. 209–222.

[14] Y. Wang and K. Yi, "Secure yannakakis: Join-aggregate queries over private data," in *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, 2021, p. 1969–1981.

[15] J. Bater, G. Elliott, C. Eggen, S. Goel, A. Kho, and J. Rogers, "Smcql: Secure querying for federated databases," *Proceedings of the VLDB Endowment*, vol. 10, no. 6, p. 673–684, 2017.

[16] S. Eskandarian and M. Zaharia, "Oblidb: oblivious query processing for secure databases," *Proceedings of the VLDB Endowment*, vol. 13, no. 2, p. 169–183, 2019.

[17] A. Arasu and R. Kaushik, "Oblivious query processing," in *Proceedings of the 17th International Conference on Database Theory*, 2014.

[18] K. E. Batcher, "Sorting networks and their applications," in *Proceedings of the AFIPS Spring Joint Computing Conference*, 1968, p. 307–314.

[19] K. Hamada, R. Kikuchi, D. Ikarashi, K. Chida, and K. Takahashi, "Practically efficient multi-party sorting protocols from comparison sort algorithms," in *Information Security and Cryptology – ICISC 2012*, 2013, pp. 202–216.

[20] G. Asharov, K. Hamada, D. Ikarashi, R. Kikuchi, A. Nof, B. Pinkas, K. Takahashi, and J. Tomida, "Efficient secure three-party sorting with applications to data analysis and heavy hitters," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, p. 125–138.

[21] G. Asharov, T.-H. H. Chan, K. Nayak, R. Pass, L. Ren, and E. Shi, "Bucket oblivious sort: An extremely simple oblivious sort," in *2020 Symposium on Simplicity in Algorithms (SOSA)*, 2020, pp. 8–14.

[22] N. Ngai, I. Demertzis, J. G. Chamani, and D. Papadopoulos, " Distributed & Scalable Oblivious Sorting and Shuffling ," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 4277–4295.

[23] Z. Chang, D. Xie, S. Wang, and F. Li, "Towards practical oblivious join," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, p. 803–817.

[24] X. Cao, W. Feng, J. Liu, J. Zhou, W. Fang, L. Wang, Q. Xu, C. Yang, and K. Ren, "Towards practical oblivious map," Cryptology ePrint Archive, Paper 2024/1650, 2024. [Online]. Available: https://eprint.iacr.org/2024/1650

[25] X. Hu and Z. Wu, "Optimal oblivious algorithms for multi-way joins," 2025. [Online]. Available: https://arxiv.org/abs/2501.04216

[26] S. Sasy and O. Ohrimenko, "Oblivious sampling algorithms for private data analysis," in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 2019.

[27] J. Bater, Y. Park, X. He, X. Wang, and J. Rogers, "Saqe: Practical privacy-preserving approximate query processing for data federations," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, p. 2691–2705, 2020.

[28] J. Bell, A. Gascón, B. Ghazi, R. Kumar, P. Manurangsi, M. Raykova, and P. Schoppmann, "Distributed, private, sparse histograms in the two-server model," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, p. 307–321.

[29] C. J. Lebeda and J. Tetek, "Better differentially private approximate histograms and heavy hitters using the misra-gries sketch," in *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, 2023, p. 79–88.

[30] J. Böhler and F. Kerschbaum, "Secure multi-party computation of differentially private median," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2147–2164.

[31] S. Zhang, S. Ray, R. Lu, Y. Guan, Y. Zheng, and J. Shao, "Secberg: Secure and practical iceberg queries in cloud," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3696–3710, 2023.

[32] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "Cryptdb: protecting confidentiality with encrypted query processing," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, 2011, p. 85–100.

[33] S. Bian, Z. Zhang, H. Pan, R. Mao, Z. Zhao, Y. Jin, and Z. Guan, "He3db: An efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, p. 2930–2944.

[34] C. Jutla and S. Patranabis, "Efficient searchable symmetric encryption for join queries," in *Advances in Cryptology – ASIACRYPT 2022*, 2022, pp. 304–333.

[35] K. Du, J. Wang, J. Wu, and Y. Wang, "Scalable equi-join queries over encrypted database," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.

[36] S. Bajaj and R. Sion, "Trusteddb: A trusted hardware-based database with privacy and data confidentiality," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 3, pp. 752–765, 2014.

[37] C. Priebe, K. Vaswani, and M. Costa, "Enclavedb: A secure database using sgx," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 264–278.

[38] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 283–298.

[39] N. Volgushev, M. Schwarzkopf, B. Getchell, M. Varia, A. Lapets, and A. Bestavros, "Conclave: secure multi-party computation on big data," in *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019.

[40] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[41] R. Bassily and A. Smith, "Local, private, efficient protocols for succinct histograms," in *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, 2015, p. 127–135.

[42] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally differentially private protocols for frequency estimation," in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Aug. 2017, pp. 729–745.

[43] G. Cormode, S. Maddock, and C. Maple, "Frequency estimation under local differential privacy," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, p. 2046–2058, Jul. 2021.

[44] Q. Luo, Y. Wang, and K. Yi, "Frequency estimation in the shuffle model with almost a single message," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, p. 2219–2232.

[45] B. Ghazi, N. Golowich, R. Kumar, R. Pagh, and A. Velingker, "On the power of multiple anonymous messages: Frequency estimation and ;selection in the shuffle model of ;differential privacy," in *Advances in Cryptology – EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part III*, 2021, p. 463–488.

[46] B. Ghazi, R. Kumar, P. Manurangsi, and R. Pagh, "Private counting from anonymous messages: near-optimal accuracy with vanishing communication overhead," in *Proceedings of the 37th International Conference on Machine Learning*, ser. ICML'20. JMLR, 2020.

[47] B. Ghazi, R. Kumar, P. Manurangsi, R. Pagh, and A. Sinha, "Differentially private aggregation in the shuffle model: Almost central accuracy in almost a single message," in *International Conference on Machine Learning*, 2021.

[48] N. Johnson, J. P. Near, and D. Song, "Towards practical differential privacy for sql queries," *Proceedings of the VLDB Endowment*, vol. 11, no. 5, p. 526–539, 2018.

[49] D. Proserpio, S. Goldberg, and F. McSherry, "Calibrating data to sensitivity in private data analysis: a platform for differentially-private analysis of weighted datasets," *Proceedings of the VLDB Endowment*, vol. 7, no. 8, p. 637–648, 2014.

[50] W. Dong and K. Yi, "Residual sensitivity for differentially private multi-way joins," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, p. 432–444.

[51] ——, "Universal private estimators," in *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, 2023, p. 195–206.

[52] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, and G. Miklau, "Privatesql: a differentially private sql query engine," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, p. 1371–1384, 2019.

[53] S. Chen and S. Zhou, "Recursive mechanism: towards node differential privacy and unrestricted joins," in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, 2013, p. 653–664.

[54] W. Dong, J. Fang, K. Yi, Y. Tao, and A. Machanavajjhala, "R2t: Instance-optimal truncation for differentially private query evaluation with foreign keys," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, p. 759–772.

[55] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum, "Differential privacy under continual observation," in *Proceedings of the Forty-Second ACM Symposium on Theory of Computing*, 2010, p. 715–724.

[56] W. Dong, Z. Chen, Q. Luo, E. Shi, and K. Yi, "Continual observation of joins under differential privacy," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, 2024.

[57] W. Dong, Q. Luo, and K. Yi, "Continual observation under user-level differential privacy," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2190–2207.

[58] Z. Jorgensen, T. Yu, and G. Cormode, "Conservative or liberal? personalized differential privacy," in *2015 IEEE 31St international conference on data engineering*. IEEE, 2015, pp. 1023–1034.

[59] H. Ebadi, D. Sands, and G. Schneider, "Differential privacy: Now it's getting personal," *Acm Sigplan Notices*, vol. 50, no. 1, pp. 69–81, 2015.

[60] D. Sun, W. Dong, Y. Qiu, and K. Yi, "Personalized truncation for personalized privacy," *Proceedings of the ACM on Management of Data*, vol. 2, no. 6, pp. 1–25, 2025.

[61] W. Dong, D. Sun, and K. Yi, "Better than composition: How to answer multiple relational queries under differential privacy," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, 2023.

[62] F. D. McSherry, "Privacy integrated queries: an extensible platform for privacy-preserving data analysis," in *Proceedings of the 2009 ACM*

SIGMOD International Conference on Management of Data, 2009, p. 19–30.

[63] J. Yu, W. Dong, J. Fang, D. Sun, and K. Yi, "Dop-sql: A general-purpose, high-utility, and extensible private sql system," *Proceedings of the VLDB Endowment*, 2024.

[64] J. Bater, X. He, W. Ehrich, A. Machanavajjhala, and J. Rogers, "Shrinkwrap: efficient sql query processing in differentially private data federations," *Proceedings of the VLDB Endowment*, vol. 12, no. 3, p. 307–320, 2018.

[65] T.-H. H. Chan, K.-M. Chung, B. Maggs, and E. Shi, "Foundations of differentially oblivious algorithms," *Journal of the ACM*, vol. 69, no. 4, 2022.

[66] L. Qin, R. Jayaram, E. Shi, Z. Song, D. Zhuo, and S. Chu, "Adore: Differentially oblivious relational database operators," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, p. 842–855, Dec. 2022.

[67] L. Qiu, G. Kellaris, N. Mamoulis, K. Nissim, and G. Kollios, "Doquet: Differentially oblivious range and join queries with private data structures," *Proceedings of the VLDB Endowment*, vol. 16, no. 13, p. 4160–4173, Sep. 2023.

[68] S. Chu, D. Zhuo, E. Shi, and T.-H. H. Chan, "Differentially oblivious database joins: Overcoming the worst-case curse of fully oblivious algorithms," in *Conference on Information-Theoretic Cryptography (ITC)*, 2021.

[69] Y. Fu and T. Wang, "Benchmarking secure sampling protocols for differential privacy," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.

[70] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Yang, X. Cheng, Y. Zhao, W. Shi, H. Xi, H. Yu, B. Liu, Y. Pan, B. Yin, J. Chen, and Q. Xu, "Oceanbase: a 707 million tpmc distributed relational database system," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, p. 3385–3397, 2022.

[71] Z. Yang, Q. Xu, S. Gao, C. Yang, G. Wang, Y. Zhao, F. Kong, H. Liu, W. Wang, and J. Xiao, "Oceanbase paetica: A hybrid shared-nothing/shared-everything database for supporting single machine and distributed cluster," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3728–3740, 2023.

[72] Q. Zhang, J. Li, H. Zhao, Q. Xu, W. Lu, J. Xiao, F. Han, C. Yang, and X. Du, "Efficient distributed transaction processing in heterogeneous network," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1372–1385, 2023.

[73] L. Qu, Q. Wang, T. Chen, K. Li, R. Zhang, X. Zhou, Q. Xu, Z. Yang, C. Yang, W. Qian *et al.*, "Are current benchmarks adequate to evaluate distributed transactional databases," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 2, no. 1, p. 100031, 2022.

[74] Q. Xu, C. Yang, and A. Zhou, "Native Distributed Databases: Problems, Challenges and Opportunities," *Proc. VLDB Endow.*, vol. 17, no. 12, p. 4217–4220, Nov. 2024. [Online]. Available: https://doi.org/10.14778/3685800.3685839

[75] F. Han, H. Liu, B. Chen, D. Jia, J. Zhou, X. Teng, C. Yang, H. Xi, W. Tian, S. Tao, S. Wang, Q. Xu, and Z. Yang, "Palf: Replicated write-ahead logging for distributed databases," *Proc. VLDB Endow.*, vol. 17, no. 12, p. 3745–3758, Nov. 2024. [Online]. Available: https://doi.org/10.14778/3685800.3685803